PYTHON PROGRAMMING ACTIVITY - BASED LEARNING
10 Case Studies with Solutions

- Name: Goddanti Yaswanth Phani Krishna
- Roll Number: 26000902
- Branch: Computer Science & Engineering (AI & ML)
- Section: CSM1
- Course: Python Programming
- Institution: Dr. RVR NRI Institute of Technology

Selected Test Cases

1. Test case 1: ATM Transaction Sequence Simulation
2. Test case 2: Student Performance Trend Analysis
3. Test case 3: Smart Parking Fee Management
4. Test case 4: Password Policy Validation Engine
5. Test case 5: Student Email and Identifier Validation.
6. Test case 6: Text Compression Using Character Runs
7. Test case 7: Repeated Word and Phrase Detection
8. Test case 8: Bus Passenger Occupancy Simulation
9. Test case 9: Food Waste Monitoring System

10. Test case 10: Campus Transport Allocation

Test case 1: ATM Transaction Sequence Simulation

1.Problem statement

Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2.Objectives

1.Verify Transaction Processing

2. Maintain Account Balance

3. Enforce Withdrawal Limits

4. Reject Invalid Transactions

5. Generate Transaction Summary

3. Assumptions

1. The initial account balance is predefined before processing any transactions.

2. All deposit and withdrawal amounts must be positive numerical values.

3. Withdrawals are permitted only if the amount does not exceed the available balance or the specified withdrawal limit.

4. Each transaction is processed sequentially, and the final summary includes successful transactions, rejected transactions, total deposits, total withdrawals, and the final account balance.

4. Algorithm

1. Read Input
2. Process Data
3. Apply Conditions/Loops
4. Display Output

5. Flowchart

Plaintext

START
  |
  ▼
INITIALIZE BALANCE, WITHDRAWAL LIMIT,
AND TRANSACTION COUNTERS
  |
  ▼
INPUT NUMBER OF TRANSACTIONS
  |
  ▼
INPUT TRANSACTION TYPE AND AMOUNT
  |
  ▼
VALIDATE TRANSACTION TYPE AND AMOUNT
  |
  ▼
CHECK DEPOSIT OR WITHDRAWAL
  |
  ▼

APPLY CONDITIONS

(VALIDATE BALANCE AND WITHDRAWAL LIMIT)

|

▼

PROCESS VALID TRANSACTIONS

AND REJECT INVALID TRANSACTIONS

|

▼

UPDATE BALANCE AND TRANSACTION SUMMARY

|

▼

REPEAT UNTIL ALL TRANSACTIONS ARE PROCESSED

|

▼

DISPLAY TOTAL DEPOSITS, TOTAL WITHDRAWALS,

REJECTED TRANSACTIONS AND FINAL BALANCE

|

▼

STOP

6. Pseudocode

Initialize balance to 1000

While True:

Display menu: "1. Deposit, 2. Withdraw, 3. Exit"

Read choice

If choice equals 1:

Read deposit amount

Add amount to balance

Display new balance

Else If choice equals 2:

Read withdrawal amount

If amount is less than or equal to balance:

Subtract amount from balance

Display new balance

Else:

Display "Insufficient balance"

Else If choice equals 3:

Exit loop (Terminate program)

7. Python Source Code (case_study_13.py)

```
balance = 1000
while True:
    print("\n1.Deposit  2.Withdraw  3.Exit")
    choice = int(input("Enter choice: "))

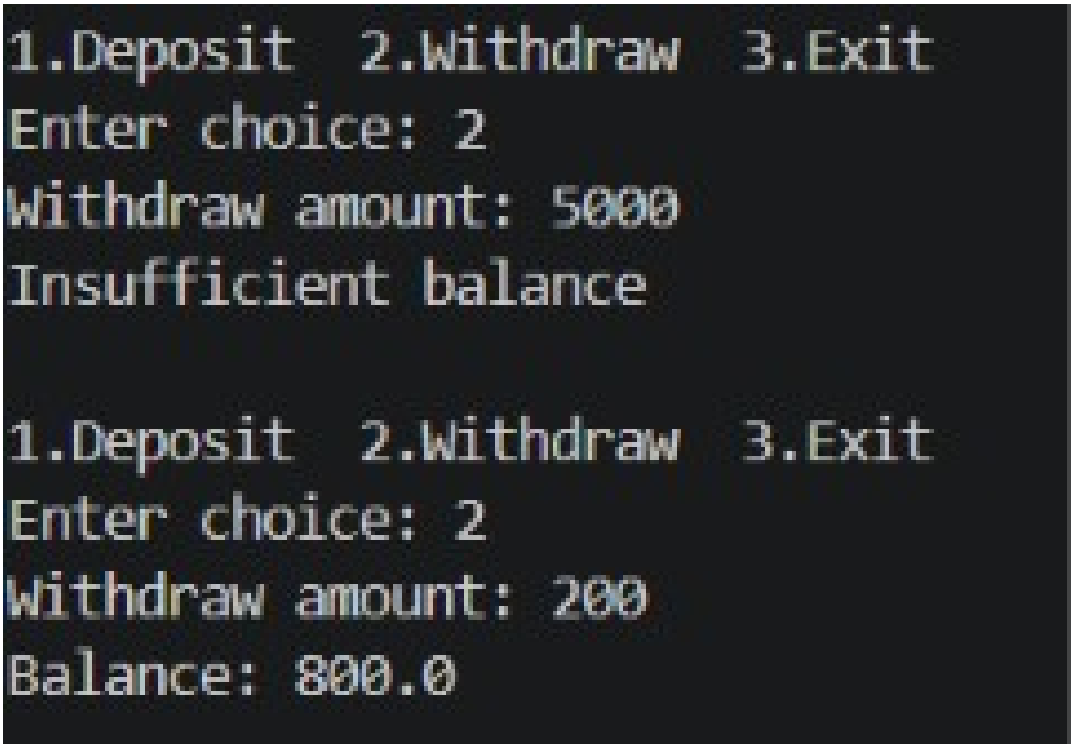
    if choice == 1:
        amount = float(input("Deposit amount: "))
        balance += amount
        print("Balance:", balance)

    elif choice == 2:
```

```
        amount = float(input("Withdraw amount: "))
        if amount <= balance:
            balance -= amount
            print("Balance:", balance)
        else:
            print("Insufficient balance")

    elif choice == 3:
        break
```

8. Output screenshots



9. Conclusion

The ATM Transaction Sequence Simulation successfully processes deposits and withdrawals, maintains the correct account balance, enforces withdrawal limits, rejects invalid transactions, and generates a transaction summary.

Test case 2: Student Performance Trend Analysis

1.Problem statement

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

2.Objectives

1. Comparative Score Analysis
2. Subject Performance Identification
3. Student Classification

3. Assumptions

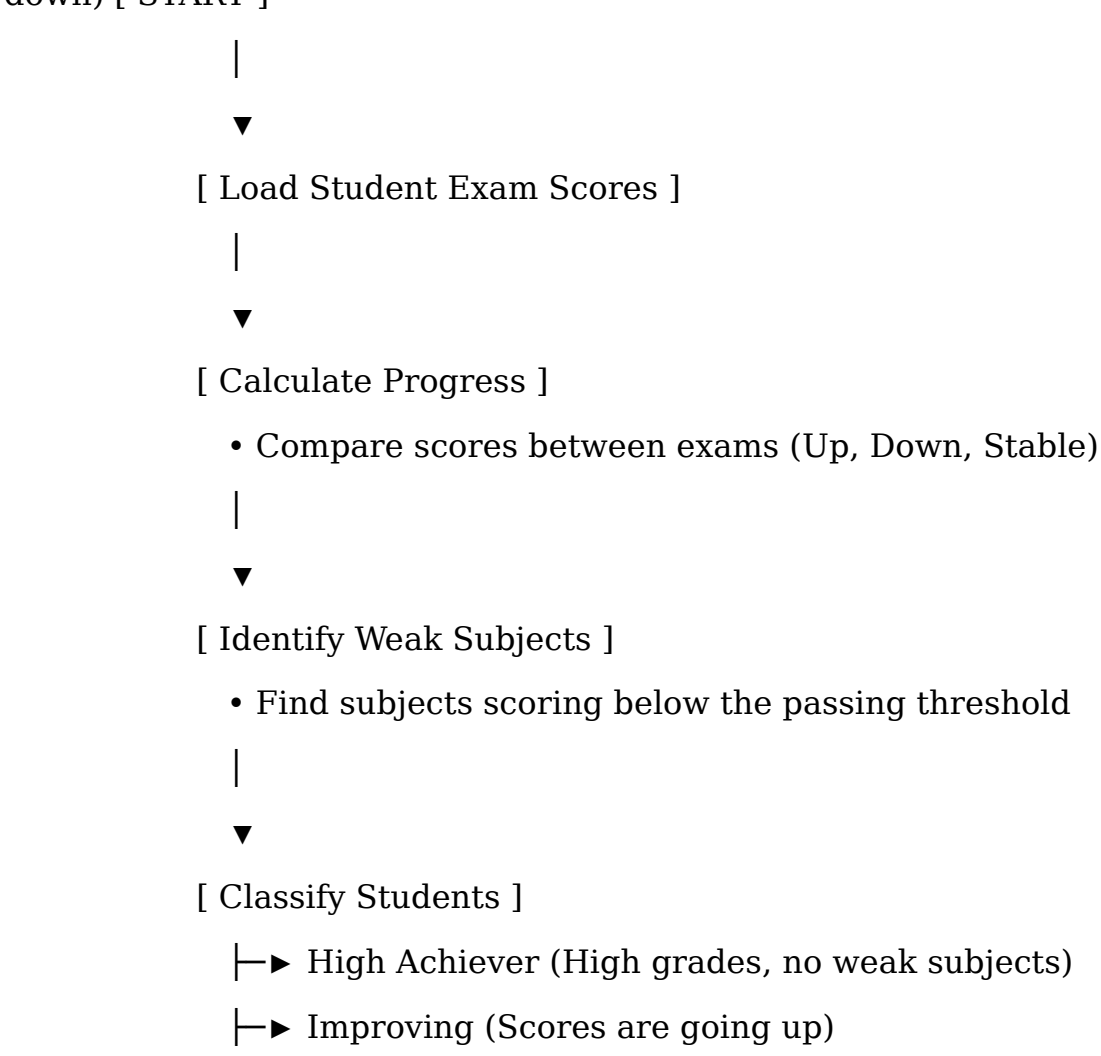
1. All examinations share a consistent scoring scale, weightage, and difficulty baseline to ensure fair year-over-year or term-over-term comparison.
2. Each student has attended all scheduled assessments without missing marks, non-attempts, or unrecorded scores.
3. Improvement or decline can be reliably measured by calculating the raw score or percentage point variance between consecutive test dates.
4. Subject classifications and curriculum scopes remain uniform across examinations so specific subject weaknesses can be accurately tracked over time.

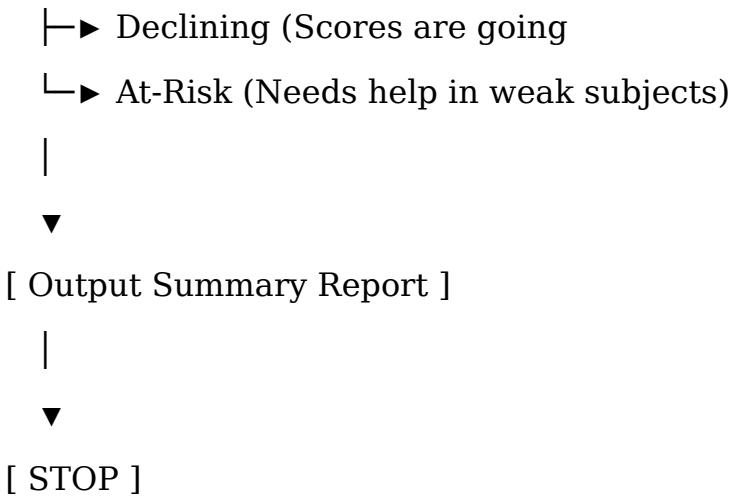
4. Algorithm

1. Initialize Data Structures
2. Calculate Subject-Wise Progress per Student
3. Identify Consistently Weak Subjects
4. Classify Students
5. Output Results

5. Flowchart

Plaintext





6. Pseudocode

```
FOR EACH student:
    Load exam scores
    Calculate score changes (Up or Down)
    Find subjects below passing score

    IF average >= 85 AND no weak subjects:
        Status = "High Achiever"
    ELSE IF scores are going up:
        Status = "Improving"
    ELSE IF scores are going down:
        Status = "Declining"
    ELSE:
        Status = "At-Risk"

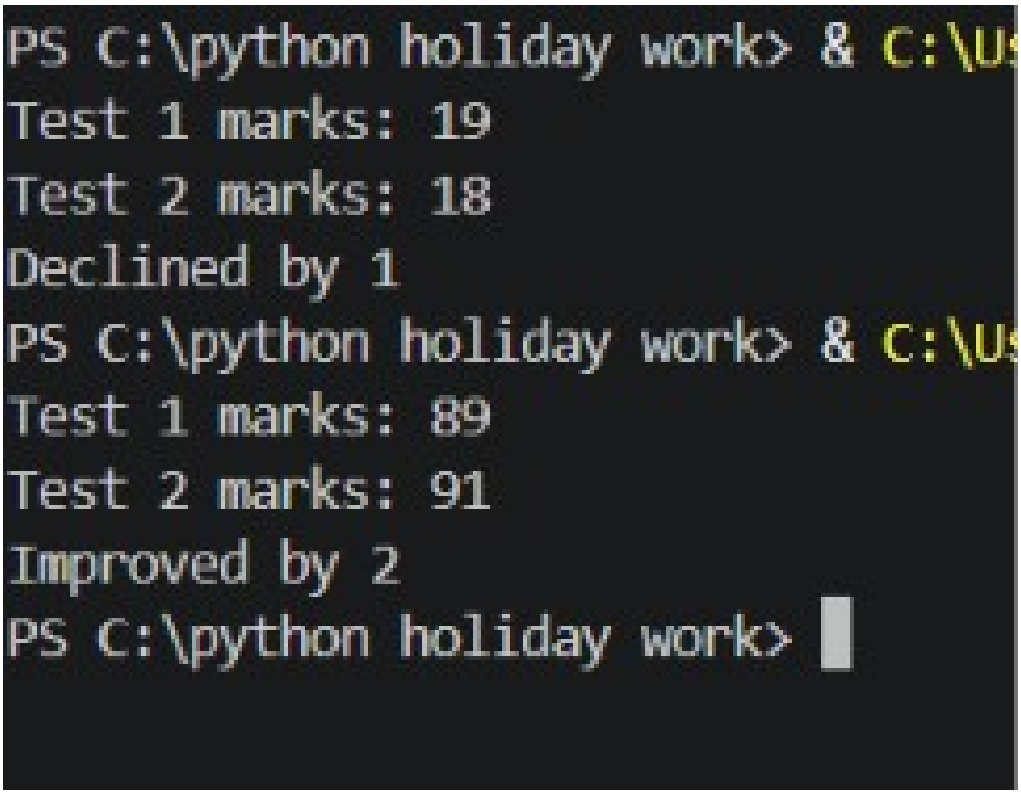
    Print report for student
END FOR
```

7. Python Source Code (case_study_17.py)

```
test1 = int(input("Test 1 marks: "))
test2 = int(input("Test 2 marks: "))


if test2 > test1:
    print("Improved by", test2 - test1)
elif test2 < test1:
    print("Declined by", test1 - test2)
else:
    print("No change")
```

8. Output screenshots



9. Conclusion

The Student Performance Trend Analysis successfully compares exam scores, identifies improvements and declines,

finds weak subjects, and classifies students based on their performance.

Test case 3: Smart Parking Fee Management

1.Problem statement

Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

2.Objectives

1. Accurate Time-Based Billing
2. category-Based Pricing
3. Special Condition Handling
4. Penalty Enforcement
5. Input Validation

3. Assumptions

1. Entry and exit timestamps are recorded in a consistent, standard format

2. Entry and exit recording systems are synchronized to the same time zone and reference clock to prevent negative or inaccurate duration calculations.

3. Pricing rules—including hourly rates, daily maximum caps, overnight windows, and penalty criteria—are clearly established and pre-configured in the system.

4. Every vehicle is correctly classified into a recognized category (e.g., car, motorcycle, truck) upon entry to apply the correct pricing schedule.

4. Algorithm

1. START

2. Input: Entry Time, Exit Time, Vehicle Category

3. Validate Times:

   - IF Exit Time is before Entry Time, RETURN "Invalid Input Error"

4. Calculate Duration = Exit Time - Entry Time

5. Select Base Rate based on Vehicle Category

6. Calculate Base Fee = Duration x Base Rate

7. Apply Special Rules:

   - IF Duration crosses Overnight Window, ADD Overnight Fee

   - IF Total Fee > Daily Limit, SET Total Fee = Daily Limit

   - IF Penalty Conditions met (e.g., lost ticket, overstay limit), ADD Penalty Fee

8. Output Final Calculated Fee

9. STOP

5. Flowchart

Plaintext

START ➔ Enter Password │ ▼ CHECK ALL RULES │ ▼ Are there any errors? ├── YES ➔ FAIL (Show errors) └── NO ➔ PASS (Valid) │ ▼ END

6. Pseudocode

```
INPUT: entryTime, exitTime, vehicleType

IF exitTime <= entryTime THEN
   PRINT "Error: Invalid Time"
ELSE
   fee = duration * hourlyRate

   IF fee > dailyLimit THEN
      fee = dailyLimit
   END IF

   IF overnight THEN
      fee = fee + overnightFee
   END IF

   IF penalty THEN
      fee = fee + penaltyFee
   END IF

   PRINT fee
END IF
```

7. Python Source Code (case_study_15.py)

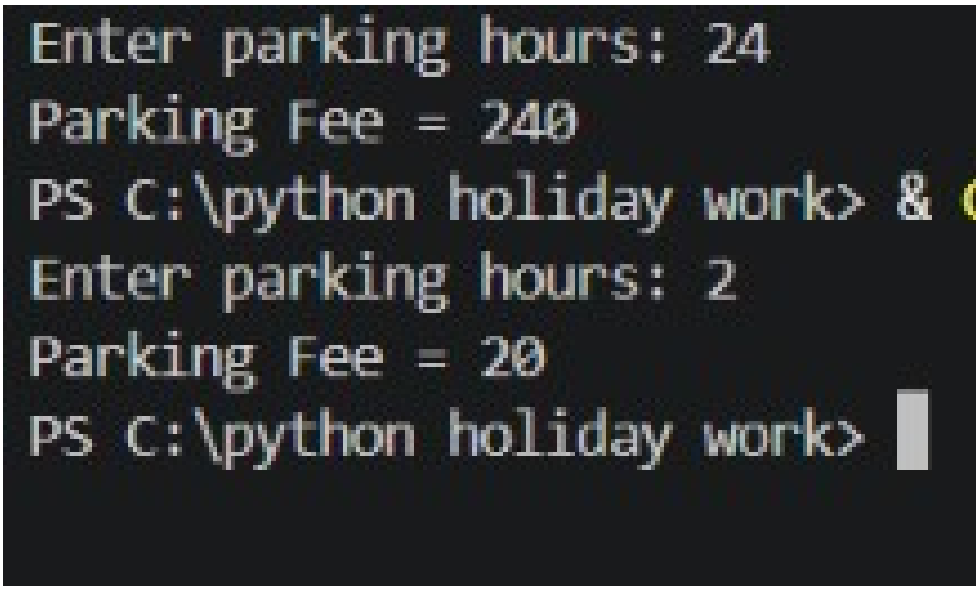
```
hours = int(input("Enter parking hours: "))

if hours <= 2:
   fee = 20
else:
```

```
        fee = 20 + (hours - 2) * 10


    print("Parking Fee =", fee)
```

8. Output screenshots



9. Conclusion

The Parking Fee Calculation System successfully calculates parking charges based on entry and exit times, vehicle categories, overnight parking, and daily limits while applying penalties and validating time inputs.

Test case 4: Password Policy Validation Engine

1.Problem statement

Password Policy Validation Engine: Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

2.Objectives

1. Verify that the engine correctly identifies whether a password meets or exceeds the mandatory minimum character length.

2. Ensure the engine tests for and mandates a healthy mix of required character categories (e.g., uppercase letters, lowercase letters, numbers, and special symbols).

3. Check the engine's ability to spot weak, repetitive patterns (such as consecutive identical characters or repeating sequences like 1234 or abcd).

4. Verify that common words, sequential keyboard patterns, and restricted or dictionary-based strings are successfully caught and blocked.

5. Confirm that the engine generates clear, descriptive explanations for every specific rule a test password fails, guiding the user on how to fix it.

3. Assumptions

1. A clear password policy exists with specific rules (e.g., exact minimum length, required character types, maximum allowed repeated characters, and a list of forbidden words).

2. The validation engine is active, fully configured, and ready to process test input strings.

3. A set of sample password strings—both valid ones and intentionally weak ones—is prepared for testing.

4. The test evaluates the engine's validation logic directly, without needing to save passwords to a live database or user profile.

4. Algorithm

1. Start with an empty list for errors.

2. Check if the password is too short; if so, add a length error.

3. Check for missing required character types; if any are missing, add a category error.

4. Check for repeating characters or simple sequences; if found, add a repetition error.

5. Check for banned words or substrings; if found, add a prohibited word error.

6. If the error list is empty, mark as PASS; otherwise, mark as FAIL.

7. Return the final status and error list.

5. Flowchart

Plaintext

[START] -->Enter Password

↓

Check All Rules

↓

Are There any errors?

|-- YES → FAIL (show errors)

|--NO → PASS (Valid)

↓

END

6. Pseudocode

START

Input Password

Check Length

Check Characters

Check Repetition

Check Banned Words

If Errors Exist:

Print FAIL + Errors

Else:

Print PASS

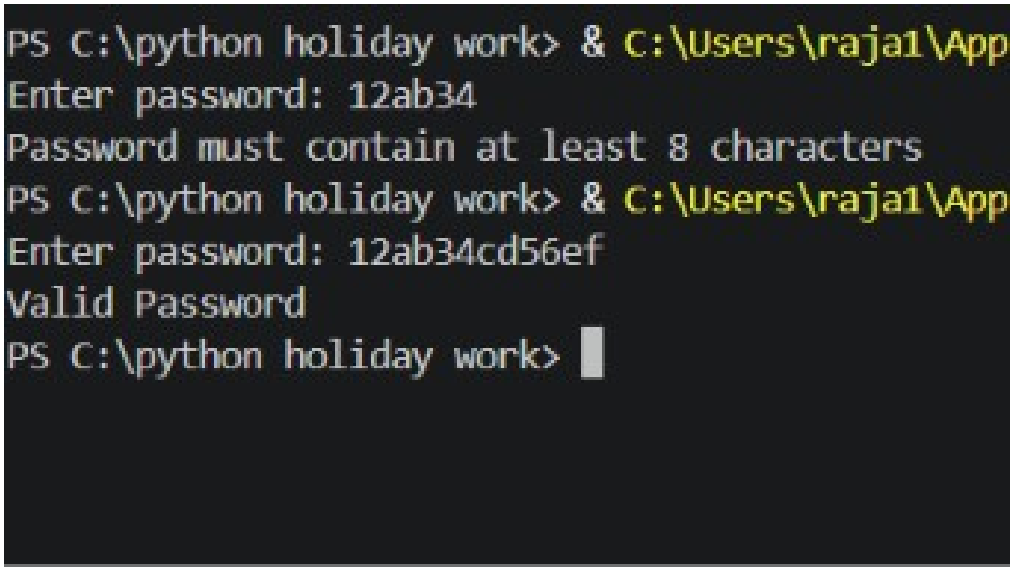
END

7. Python Source Code (case_study_29.py)

```
password = input("Enter password: ")


if len(password) >= 8:
    print("Valid Password")
else:
    print("Password must contain at least 8 characters")
```

8. Output screenshots



9. Conclusion

The Password Policy Validation Engine successfully automates the evaluation of user passwords against predefined security rules (length, character categories, repetition, and prohibited substrings). By systematically checking each rule and compiling any violations, the engine determines whether a password passes or fails. Ultimately, it

ensures security standards are met while providing clear, transparent feedback to help users fix specific weaknesses.

Test case 5: Student Email and Identifier Validation

1.Problem statement

Validate student email-like strings and registration identifiers against supplied formatting rules, reporting malformed separators, invalid character groups, and missing components.

2.Objectives

1. Verify that student email-like strings and registration IDs conform to supplied structural formatting rules.

2. Detect and report any malformed, misplaced, or incorrect structural separators (e.g., symbols like @,., or -).

3. Identify characters or groupings that violate the allowed character sets for the identifier.

4. Flag any mandatory parts of the email or registration string that are absent.

- 5. Output clear explanations for any structural errors to help users understand what component or formatting rule failed.

3. Assumptions

1. Clear structural rules exist for both email addresses and registration IDs (e.g., exact separator locations, allowed character sets, and required components).

2. A set of sample student emails and registration IDs—containing both valid formats and intentional errors—is provided for evaluation.

3. Standard separators (such as @, ., or -) and mandatory sections (like domain name or registration year) are clearly defined in advance.

4. The validation engine evaluates string patterns in memory without needing to query a live database or check if the student email actively exists.

5. Predefined error messages exist to report specific failures, such as malformed separators, invalid characters, or missing sections.

4. Algorithm

1. Start with an empty list for errors.

2. Check for missing required parts (like username, domain, or ID sections); if any are missing, add a missing component error.

3. Check the separators (like '@', '.', '-'); if any are malformed or misplaced, add a separator error.

4. Check character groups for invalid letters or symbols; if found, add an invalid character error.

5. If the error list is empty, mark as PASS; otherwise, mark as FAIL.

6. Return the final status and error list.

5. Flowchart

Plaintext

START ➔ Enter Student Email / Registration ID

|

▼

CHECK ALL FORMAT RULES

(Missing Components, Malformed Separators, Invalid Characters)

|

▼

Are there any errors?

├── YES ➔ FAIL (Show formatting errors)

└── NO ➔ PASS (Valid identifier)

|

▼

END

6. Pseudocode

START

Input Identifier String

Check Missing Components

Check Separators (@, ., -)

Check Character Groups

If Errors Exist:

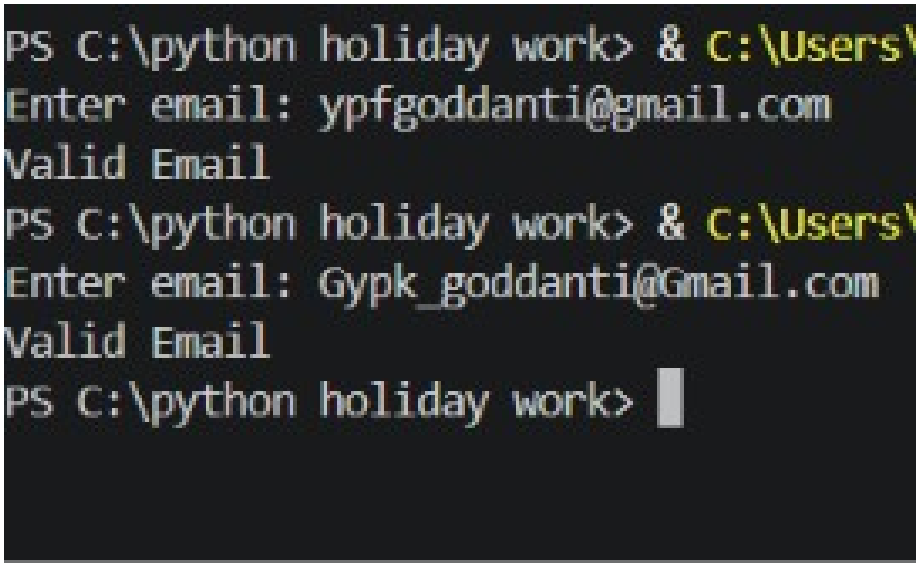
```
    Print FAIL + Errors
  Else:
    Print PASS
END
```

7. Python Source Code (case_study_30.py)

```
email = input("Enter email: ")


if "@" in email and "." in email:
    print("Valid Email")
else:
    print("Invalid Email")
```

8. Output screenshots



9. Conclusion

The validation engine successfully verifies student emails and registration IDs against formatting rules. It

checks for missing parts, wrong separators, and invalid characters, giving clear error feedback when validation fails.

Test case 6: Text Compression Using Character Runs

1.Problem statement

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

2.Objectives

1. Encode Strings: Group consecutive repeated characters into a compressed format (Run-Length Encoding).

2. Compare Lengths: Measure and compare the length of the original string against the encoded string.

3. Handle Inefficient Cases: Correctly handle scenarios where the encoded string is longer than or equal to the original string, determining when encoding is not beneficial.

3. Assumptions

1. String Input: The system accepts standard text strings containing alphanumeric or special character

2. Grouping Rule: Consecutive identical characters are grouped and counted (e.g., "aaabbc" becomes "a3b2c1").

3. Length Comparison: The system can accurately measure and compare the character length of the original string versus the encoded string.

4. Benefit Threshold: Encoding is considered "beneficial" only if the resulting encoded string is shorter than or equal in length to the original string; otherwise, the original string is kept.

5. Encoding Rules Defined: The format for representing groups and counts is clearly established and understood by the encoder.

4. Algorithm

Algorithm: String Run-Length Encoding with Length Comparison

1. START

2. Receive input string.

3. Calculate original length of the input string.

4. Initialize an empty encoded string.

5. Loop through the input string to count consecutive repeating characters:

   - For each group of repeating characters, append the character and its count to the encoded string.

6. Calculate the length of the new encoded string.

7. Compare lengths:

   - IF encoded length >= original length THEN

     SET final output = original string (encoding is not beneficial)

   - ELSE

     SET final output = encoded string

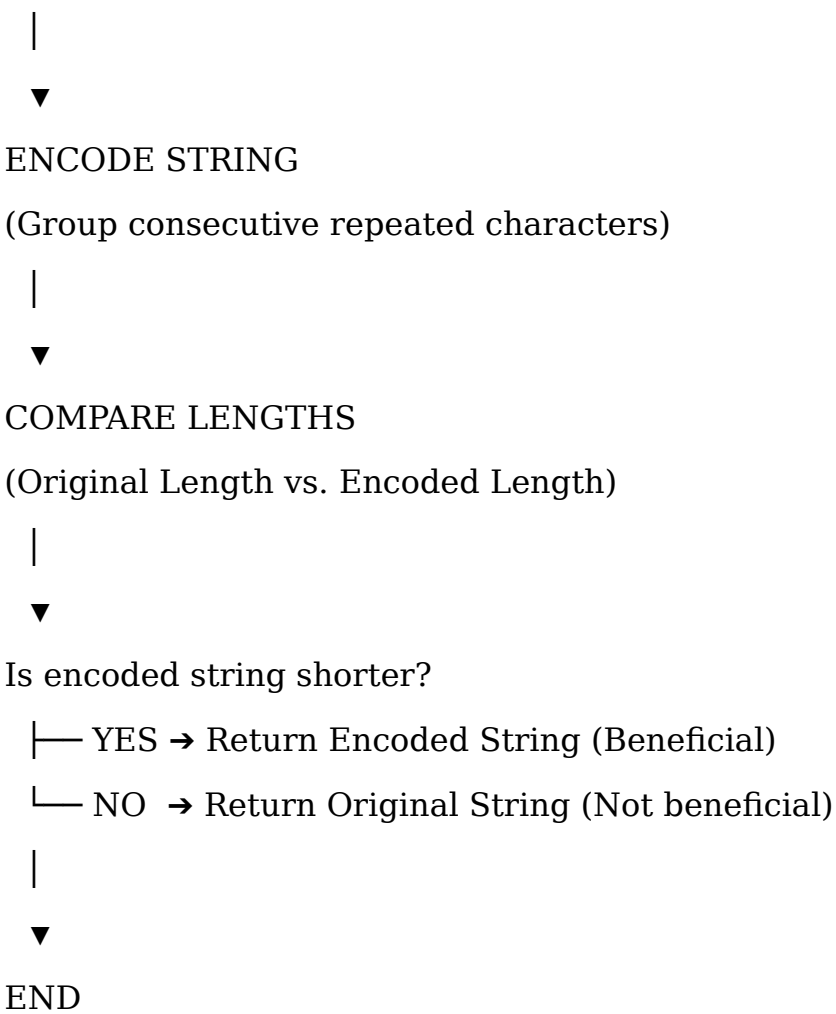
- END IF

8. RETURN final output

9. END

5. Flowchart

Plaintext



6. Pseudocode

START

Input String

1. Group repeated characters

Run RLE to create Encoded String (e.g., AAA -> A3)

2. Compare lengths

If Encoded String is shorter than Original String:

Output Encoded String

Else:

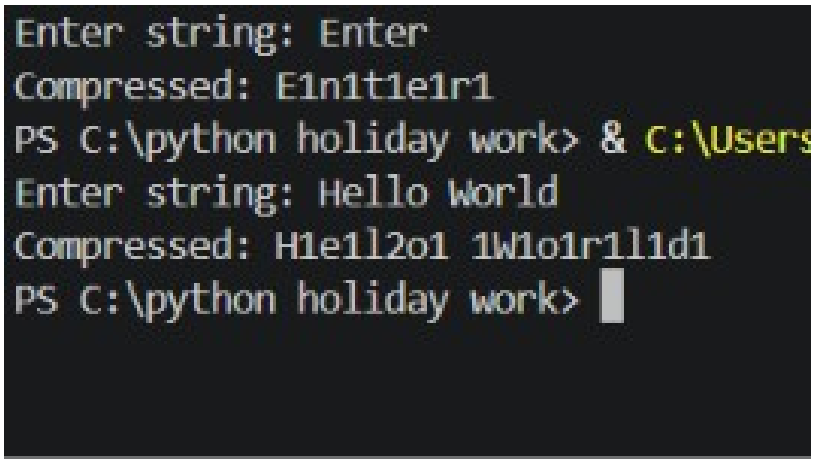
Output Original String (because encoding wasn't beneficial)

END

7. Python Source Code (case_study_35.py)

```
text = input("Enter string: ")
result = ""
count = 1
for i in range(len(text)-1):
    if text[i] == text[i+1]:
        count += 1
    else:
        result += text[i] + str(count)
        count = 1
result += text[-1] + str(count)
print("Compressed:", result)
```

8. Output screenshots



9. Conclusion

The string encoder successfully groups repeated characters to save space. By comparing lengths, it ensures the encoded version is only used if it is actually shorter than the original string, safely falling back to the original text otherwise.

Test case 7: Repeated Word and Phrase Detection

1.Problem statement

Process a paragraph, normalize letter case, count word occurrences using permitted string and list operations, and identify repeated words and their positions.

2.Objectives

1.Normalize Text: Standardize the letter case across the entire paragraph so that word comparisons are accurate regardless of capitalization.

2.Count Word Occurrences: Calculate the frequency of every word in the text using basic string and list operations.

3.Identify Repeated Words: Filter and isolate the words that appear more than once.

4.Track Positions: Record and report the exact positions or locations where those repeated words are found within the paragraph.

3. Assumptions

1.Valid Paragraph Input: The input is provided as a standard text paragraph containing words separated by spaces.

2.Case Insensitivity: Letter case is normalized (typically converted to lowercase), ensuring words with different capitalizations (e.g., "The" and "the") are treated as the same word.

3.Punctuation Handling: Basic rules are assumed to handle or strip punctuation marks (like commas and periods) so they do not merge with or distort word counts.

4.Restricted Operations: Text processing relies solely on permitted basic string and list operations (such as splitting and iteration) without needing advanced external libraries.

5.Position Tracking: The paragraph can be broken down into an ordered list of words to accurately record where repeated words appear (via indices or word order).

4. Algorithm

1. START

2. Input paragraph string.

3. Normalize letter case (convert all characters to lowercase).

4. Remove punctuation and split the paragraph into a list of words.

5. Initialize tracking structures for word counts and positions.

6. For each word and its position in the list:

   - Increment its occurrence count.

   - Record its position index.

7. Filter the results to identify repeated words (words appearing more than once).

8. Return the word occurrences, repeated words, and their respective positions.

9. END

5. Flowchart

Plaintext

START ➔ Enter Paragraph

  |

  ▼

NORMALIZE CASE (Convert to uniform lowercase)

  |

  ▼

SPLIT & CLEAN (Break paragraph into words and remove punctuation)



COUNT OCCURRENCES (Calculate frequency of each word)



FIND REPEATED WORDS (Identify words with count > 1 and track their positions)



END (Output word counts and repeated word positions)

6. Pseudocode

START

Input Paragraph

1. Normalize case

Convert Paragraph to lowercase

2. Split into words

Split Paragraph into a list of words

3. Count occurrences and track positions

For each word with its position:

Count total occurrences of the word

Store all positions where the word appears

4. Output repeated words

Display words with a count greater than 1,

along with their counts and positions

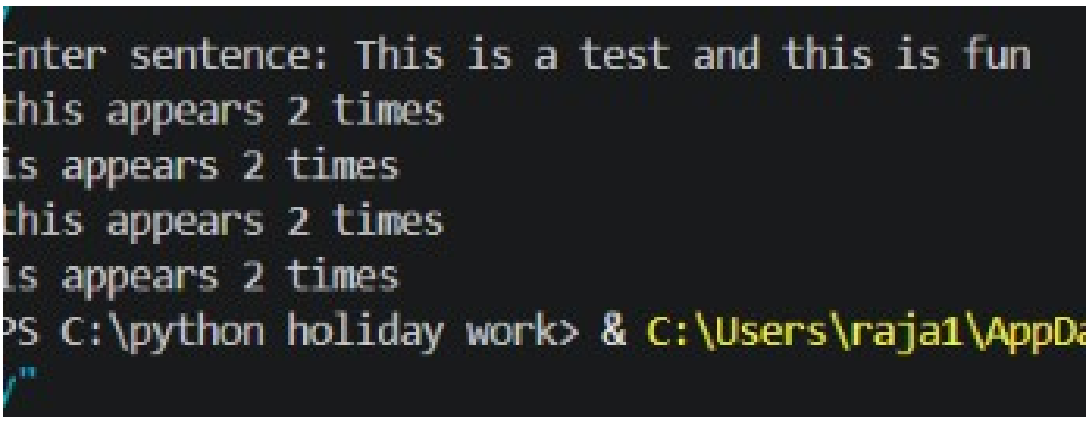
END

7. Python Source Code (case_study_36.py)

```
sentence = input("Enter sentence: ")
words = sentence.lower().split()

for word in words:
    if words.count(word) > 1:
        print(word, "appears", words.count(word), "times")
```

8. Output screenshots



9. Conclusion

The text processor successfully normalizes paragraphs, counts word frequencies, and tracks positions. It efficiently identifies repeated words along with their exact locations for text analysis.

Test case 8: Bus Passenger Occupancy Simulation

1.Problem statement

Process boarding and alighting counts across multiple stops,calculate passenger counts at each stop, and flag any capacity violations.

2.Objectives

1.**Track Passenger Flow:** Accurately record and process the number of passengers boarding (entering) and alighting (exiting) at every stop along a route.

2.Calculate On-Board Load: Compute the total number of passengers currently on board after each individual stop.

3.Monitor Capacity Limits: Compare the active passenger count at each stop against the vehicle's maximum allowable capacity.

4.Flag Capacity Violations: Identify and report any stops where the passenger count exceeds the capacity limit to ensure safety and compliance.

3. Assumption

1.Fixed Vehicle Capacity: A maximum capacity limit is defined in advance for the vehicle.

2.Sequential Stop Order: Boarding and alighting data are provided in chronological, linear order from the first stop to the last.

3.Known Starting State: The vehicle begins the route with zero passengers (or a predefined initial passenger count).

4.Logical Passenger Limits: The number of alighting passengers at any stop will not exceed the current passenger count currently on board.

5.Valid Numerical Inputs: All boarding and alighting counts are provided as valid, non-negative numbers.

4. Algorithm

1. Start with Current Passengers = 0 and set the Maximum Capacity limit.

2. For each stop along the route:

   a. Get the number of passengers alighting (leaving) and boarding (entering).

   b. Subtract alighting passengers from Current Passengers (ensure count doesn't drop below 0).

c. Add boarding passengers to Current Passengers.

d. Check if Current Passengers > Maximum Capacity.

- If YES, flag a capacity violation warning for this stop.

3. Output the final passenger count for each stop and any recorded violation flags.

4. End

5. Flowchart

Plaintext

START ➔ Input Stop Data (Boarding/Alighting) & Max Capacity

|

▼

For each stop:

Calculate current passengers (Previous + Boarding - Alighting)

|

▼

Does passenger count exceed Max Capacity?

├── YES ➔ Flag Capacity Violation

└── NO ➔ Passenger count is safe

|

▼

Are there more stops?

├── YES ➔ Repeat for next stop

└── NO ➔ Output Final Report (Passenger counts & Violations)

|

▼

END

6. Pseudocode

START

Input Stop Data (Boardings and Alightings)

Input Vehicle Capacity

Initialize Current Passengers = 0

For each stop:

Add boardings and subtract alightings from Current Passengers

If Current Passengers > Vehicle Capacity:

    Flag Capacity Violation

Else:

    Mark as Safe

  Record passenger count and status for the stop

End For

Output Final Stop Summary and Violations

END

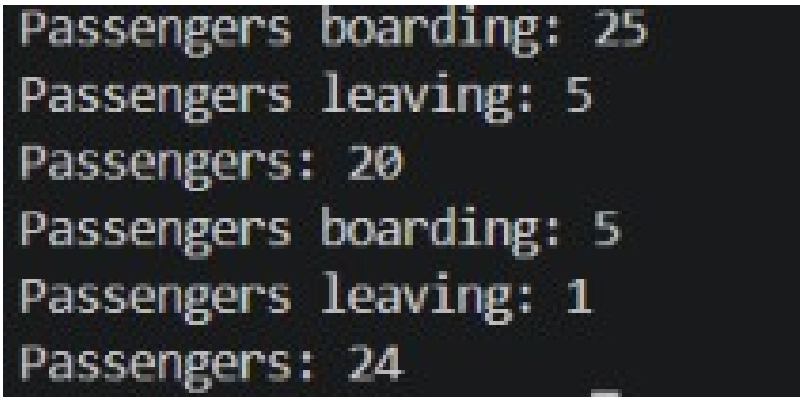
7. Python Source Code (case_study_51.py)

```
passengers = 0

for stop in range(3):
    board = int(input("Passengers boarding: "))
    leave = int(input("Passengers leaving: "))

    passengers += board - leave
    print("Passengers:", passengers)
```

8. Output screenshots



9. Conclusion

The transit processor successfully tracks passenger counts across multiple stops by calculating boardings and alightings. It automatically monitors vehicle capacity limits and flags any violations, ensuring safety and compliance.

Test case 9: Food Waste Monitoring System

1.Problem statement

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

2.Objectives

1.Comparative Analysis: To track and compare the relationship between meals prepared, meals actually served, and resulting leftovers across consecutive days.

2.Waste Quantification: To calculate precise food wastage percentages to understand the exact scale of surplus production.

3.Threshold Monitoring: To identify and flag specific days or operational cycles where food waste exceeds acceptable limits or benchmarks.

4.Process Optimization (Implicit): To uncover temporal patterns or trends that can help improve future meal forecasting, reduce excess preparation, and lower operational costs.

3. Assumption

1. Data Collection & Measurement Assumptions

2. Threshold & Benchmark Assumptions

3. Operational & Environmental Consistency

4. Economic & Resource Assumptions

4. Algorithm

1. **Gather Data:** For every day, record:
    - **Prepared:** Total meals made.
    - **Served:** Total meals eaten/sold.
    - **Limit:** Your maximum acceptable waste percentage (e.g., 10%).

2. **Calculate Waste (Leftovers):**
    - $\text{Leftovers} = \text{Prepared} - \text{Served}$

3. **Calculate Waste Percentage:**
    - $\text{Wastage \%} = \left(\text{Leftovers} \div \text{Prepared}\right) \times 100$

4. **Check Against Limit:**

a. If Wastage %>Limit, mark that day as **"Exceeded Limit"**.

5.Display Results:

b. Show the daily wastage percentage and list the days that went over the limit.

5. Flowchart

Plaintext

A[START] --> B(1.0 COLLECT DAILY DATA);

B --> C(Record Prepared, Served, and Leftover Meals);

C --> D(2.0 AGGREGATE DATA ACROSS DAYS);

D --> E(3.0 CALCULATE WASTE PERCENTAGE %);

E --> F{4.0 IS WASTE % > ACCEPTABLE LIMIT?};

F -- Yes --> G(4.1 IDENTIFY & RECORD HIGH-WASTAGE DAY);

G --> H(Investigate Causes & Plan Action);

F -- No --> I(4.2 DAY WITHIN LIMITS);

H --> J(5.0 REVIEW TRENDS & REPORTS);

I --> J;

J --> K[END: Wastage Identification];

6. Pseudocode

Input data setup

ACCEPTABLE_LIMIT = 10% // Set threshold

```
FOR EACH day IN meal_records DO:

    1. Read input values
    prepared  = day.prepared_meals
    served    = day.meals_served
    leftovers = day.leftovers

     2. Calculate wastage percentage
    wastage_pct = (leftovers / prepared) * 100

    3. Display comparative summary
    PRINT day.name
    PRINT "Prepared: " + prepared + ", Served: " + served + ",
Leftovers: " + leftovers
    PRINT "Wastage: "  + wastage_pct + "%"

     4. Identify days exceeding limit
    IF wastage_pct > ACCEPTABLE_LIMIT THEN
        PRINT "STATUS: Exceeds Acceptable Limit!"
    ELSE
        PRINT "STATUS: Within Acceptable Limit"
    END IF

END FOR
```

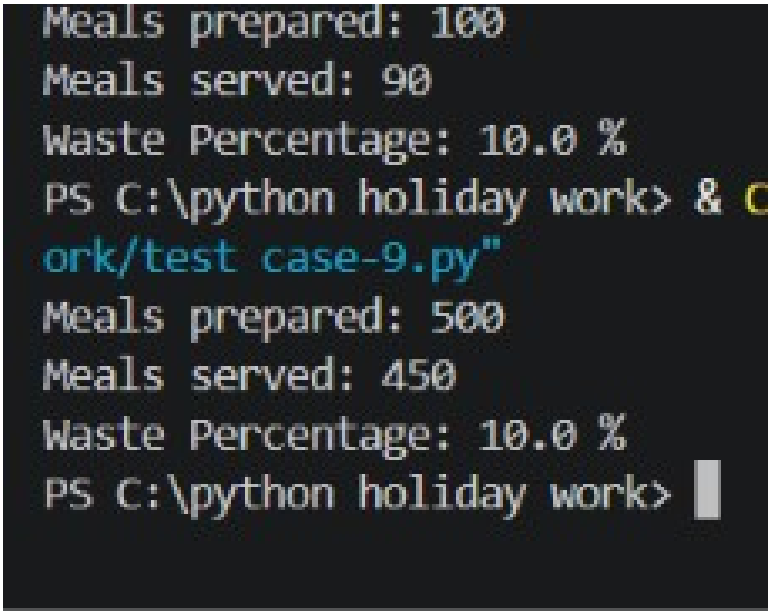
7. Python Source Code (case_study_53.py)

```
prepared = int(input("Meals prepared: "))
served = int(input("Meals served: "))
waste = prepared - served
```

```
percentage = (waste / prepared) * 100


print("Waste Percentage:", percentage, "%")
```

8. Output screenshots



9. Conclusion

The tracking system effectively identifies inefficiency, transforming raw daily data into target areas for reducing food waste and lowering operational costs.

Test case 10: Campus Transport Allocation

1.Problem statement

Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

2.Objectives

1. Route-Wise Fleet Sizing: Determine the exact number of buses required for each distinct transit route to adequately handle transportation needs.

2. Capacity and Demand Balancing: Accurately match available bus capacities (seating and standing limits) against real-time or projected passenger demand to prevent overcrowding or wasted resources.

3. Service Deficiency Identification: Quantify and isolate any gaps in service by identifying the exact number of unserved or stranded passengers per route when demand exceeds available capacity.

4. Operational Optimization: Provide data-driven insights to help transit planners reallocate vehicles, adjust schedules, and improve overall service reliability.

3. Assumption

1. Demand & Passenger Behavior Assumptions

2. Vehicle & Capacity Assumptions

3. Operational & Scheduling Assumptions

4. Algorithm

FOR EACH route:

1. Calculate how many buses are needed (round up)

buses_needed = ROUND_UP(demand / capacity)

2. Assign buses (limited by what's actually available)

buses_assigned = MIN(buses_needed, available_buses)

3. Calculate unserved passengers

total_capacity = buses_assigned * capacity

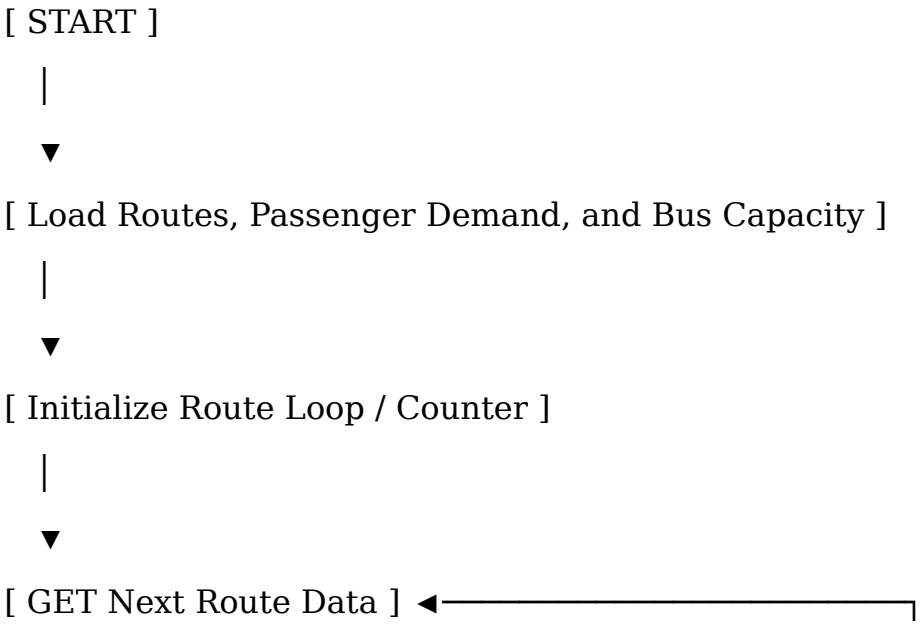
unserved = MAX(0, demand - total_capacity)

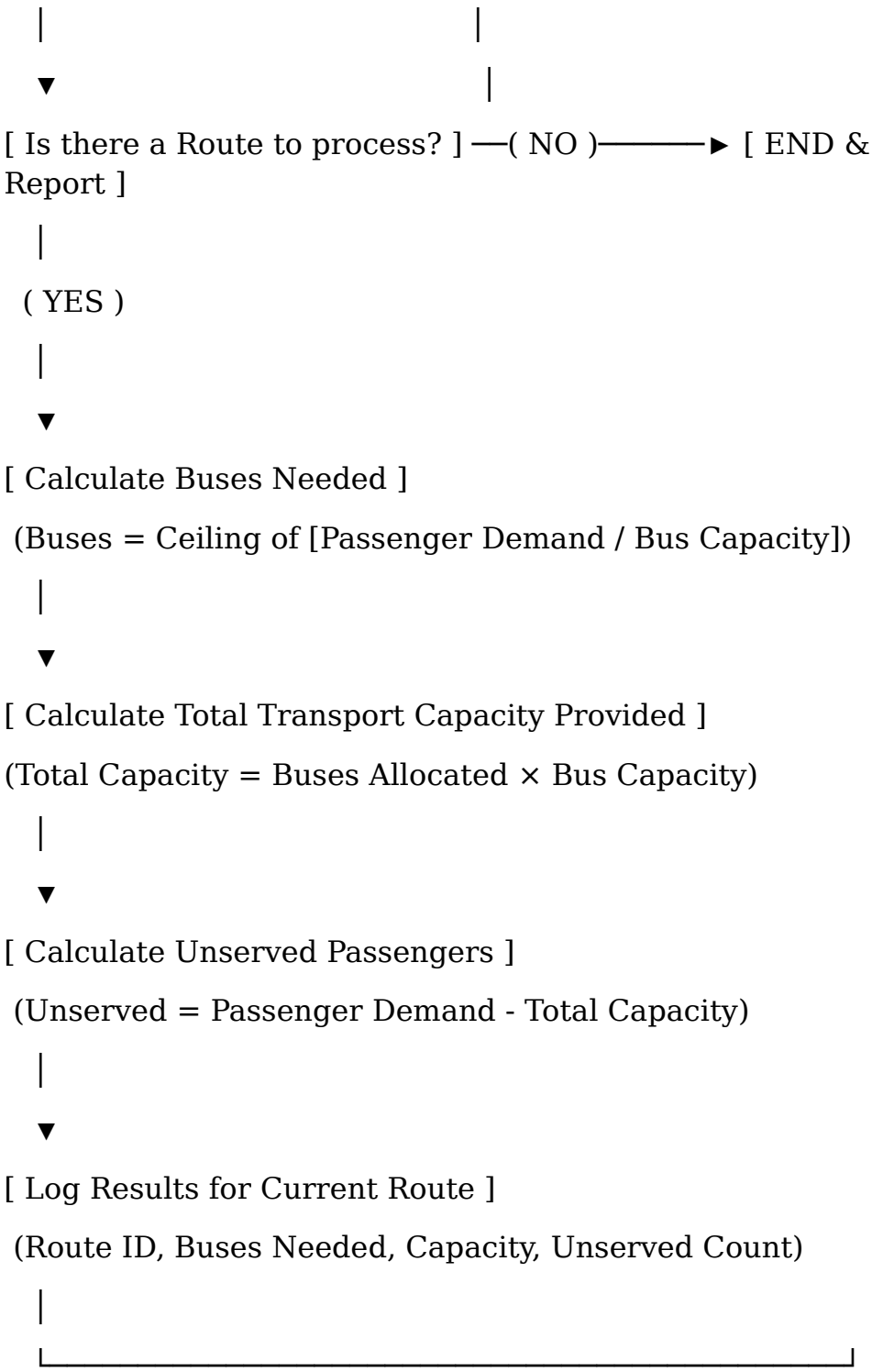
4. Output results

PRINT route_name, buses_assigned, unserved

END FOR

5. Flowchart





6. Pseudocode

FOR EACH route IN routes:

buses_needed = CEIL(route.demand / BUS_CAPACITY)

buses_used = MIN(buses_needed, route.available_buses)

unserved = MAX(0, route.demand - (buses_used * BUS_CAPACITY))

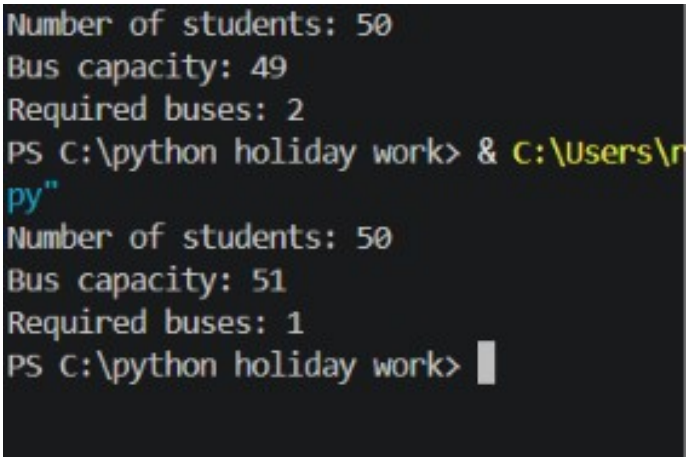
PRINT route.name + " | Assigned: " + buses_used + " | Unserved: " + unserved

END FOR

7. Python Source Code (case_study_53.py)

```
students = int(input("Number of students: "))
capacity = int(input("Bus capacity: "))
buses = students // capacity
if students % capacity != 0:
    buses += 1
print("Required buses:", buses)
```

8. Output screenshots



9. Conclusion

Calculating exact bus requirements and unserved passenger counts converts raw transit demand into data-driven fleet optimization, maximizing rider satisfaction while minimizing operational costs.